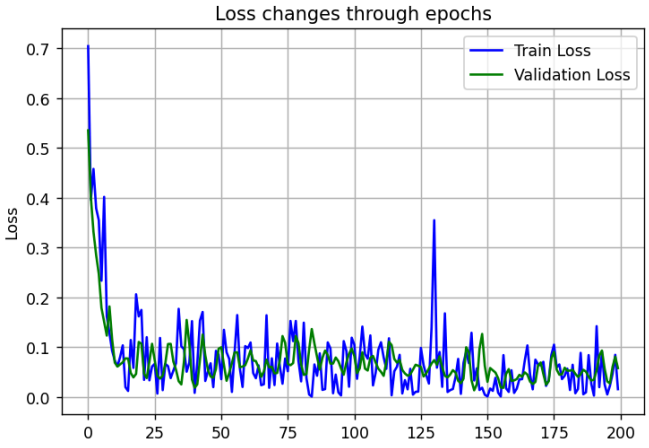
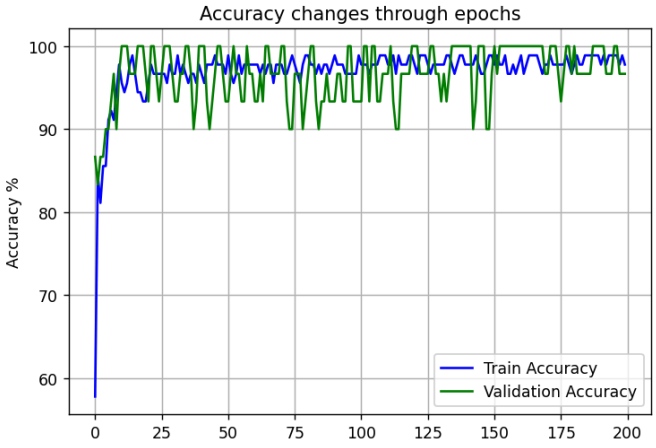
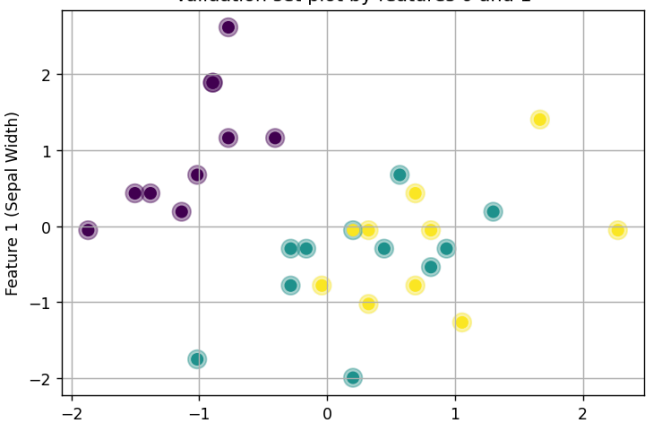
Neural Network Training Diagnosis

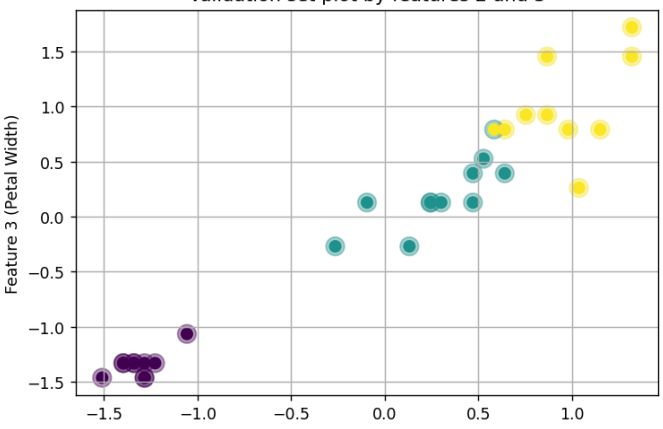
(Kyryliuk Ivan, s32785)

Task: Classification
Dataset: Iris, split size 90/30/30, sampled (4 features, 3 classes)
Hyperparameters: Number of Neurons on hidden layer, Number of Epochs, Learning rate, Batch size, Weight Decay, Dropout Probability, Optimizer.
Final Model Settings: One Hidden Layer, 16 neurons on hidden layer, 200 epochs, lr = 0.05, Batch size = 30, Weight Decay = 1e-3, Dropout Probability = 0.10,
Optimizer = AdamW (Compatible with weight decay).
Activation functions: Linear on outer layer, RELU on hidden layer.







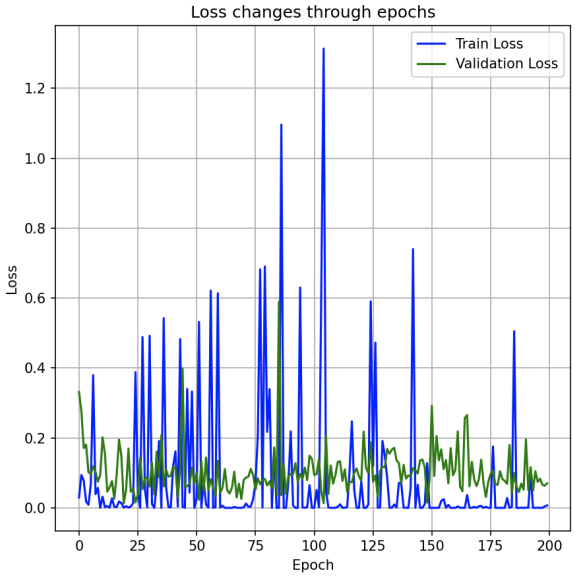


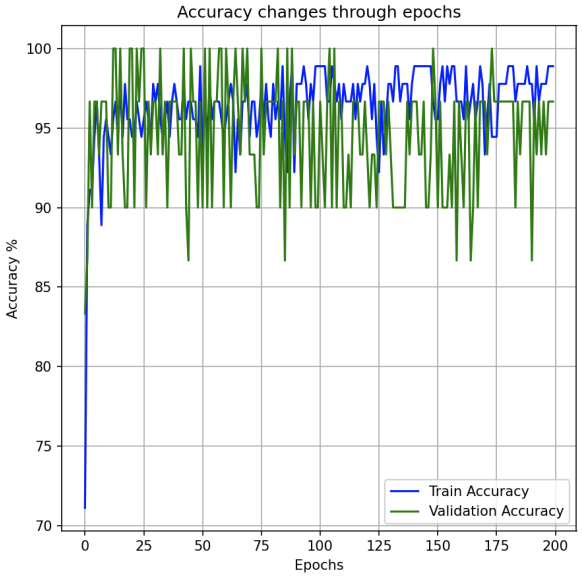
Final model configuration and results

- Such final configuration was chosen because it is a middle ground between training stability and compatibility for any hardware. The batch size is set accordingly, and a balanced learning rate prevents overfitting and underfitting.
- Dropout probability is set to 10 percent to not cause too much training noise.
- As optimizer function I chose AdamW, as Adam uses a great concept of momentum + W part makes it very well compatible with Weight Decay.
- Such high number of epochs was chosen to better analyze concepts of underfitting and overfitting.
  As measurements I used standard Loss and Accuracy, showcased on the graphs. These metrics fit well for classification tasks.
- As a result model achieved 0.0155 Training and 0.0578 Validation Loss with 100 percent Test set accuracy.
- Possible improvement is adding learning rate decay, as current setup achieves great accuracy and loss fast and then varies about these values for next 100 epochs.

Problems and solutions

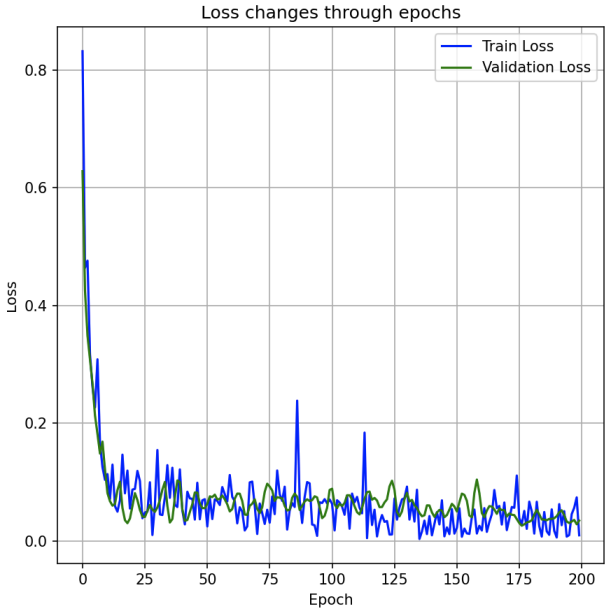
Unstable training: With small batch sizes the training samples vary drastically, producing big Loss changes, leading to an unstable training process. *Measures for batch size of 3*:

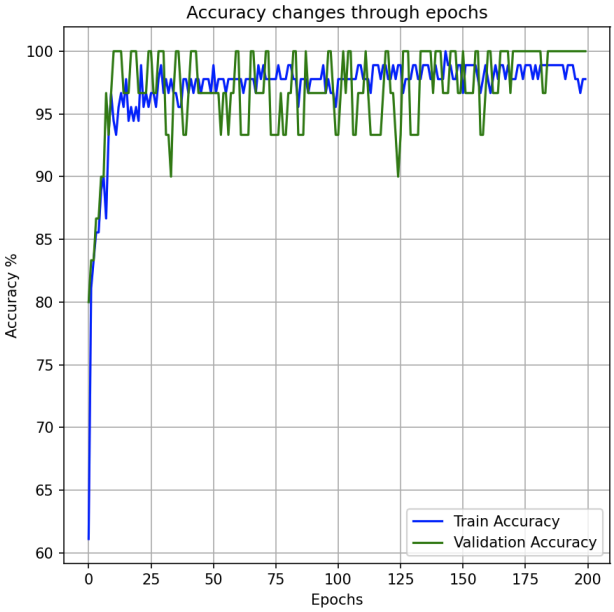




The solution to such problem is increasing batch size to reasonable sizes. Another solution could be decreasing learning rate (starting with higher values to explore the dataset faster and then lowering it to find minimum with better approximation and less noise)

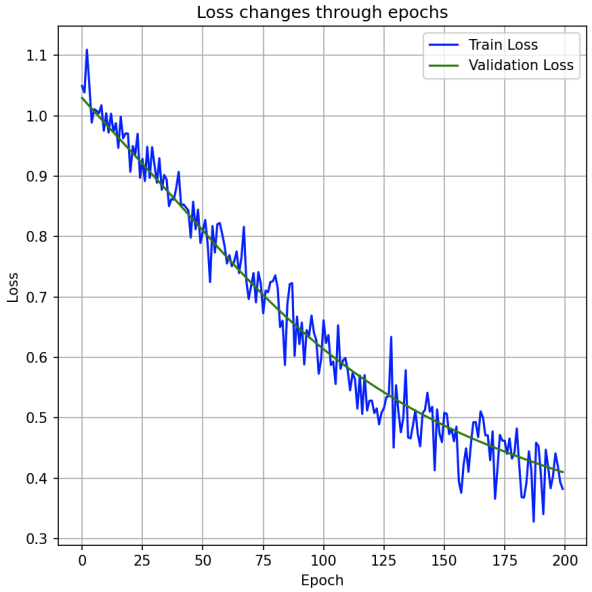
Measures for batch size of 48:

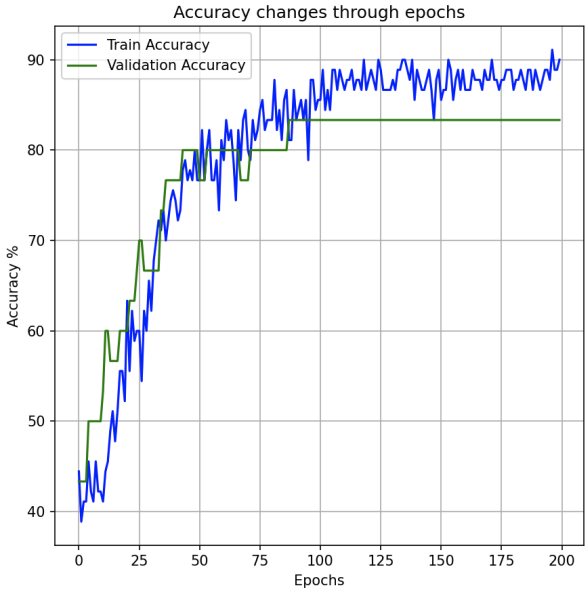




Underfitting: Learning rate too small leads to slow training process.

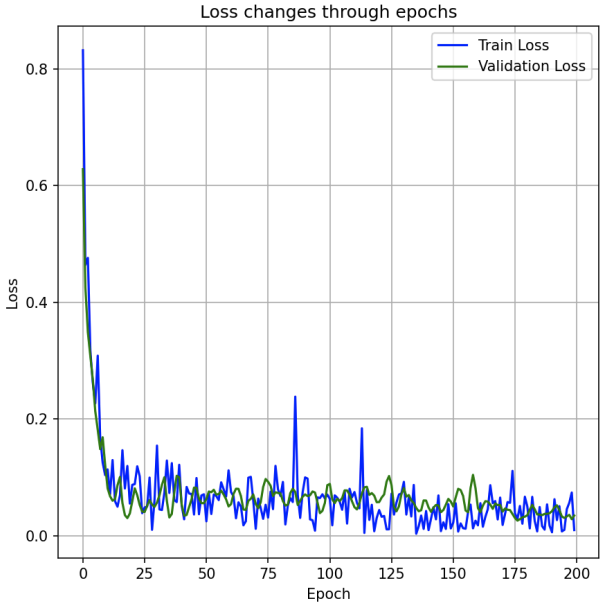
Lr = 0.0005



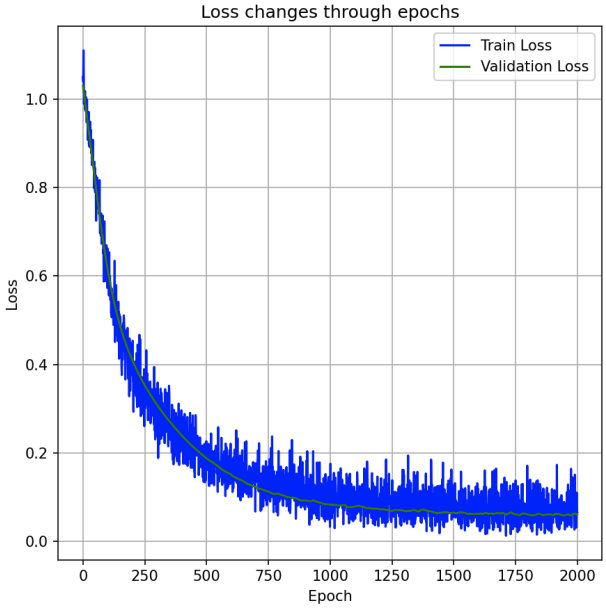


Possible solutions may be: Increasing Lr or Increasing number of epochs

Lr = 0.05



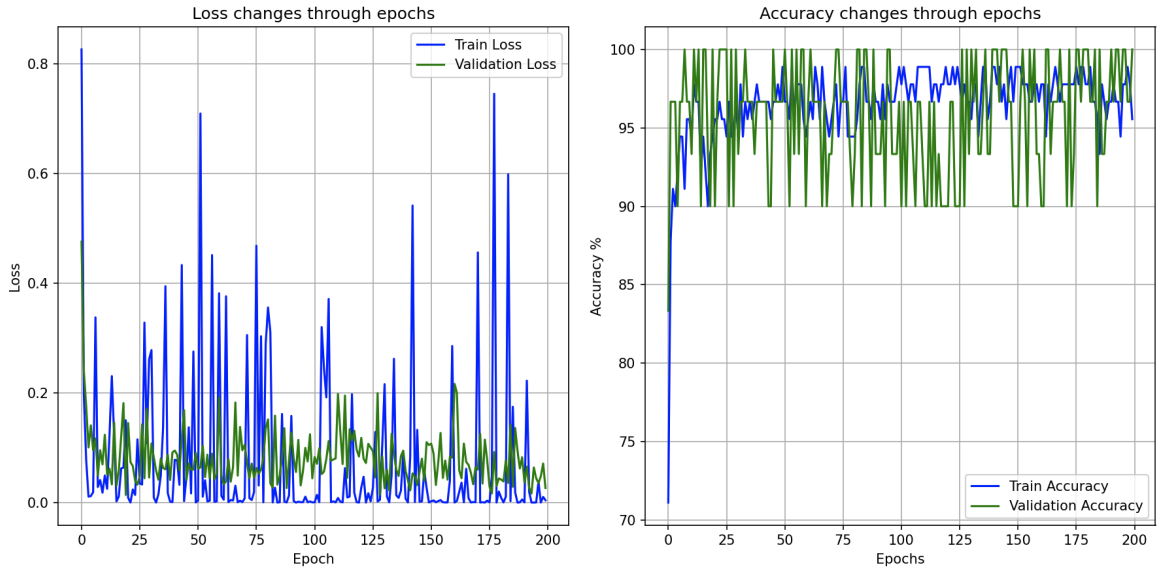
Lr = 0.0005, N_Epochs = 2000



Experiment

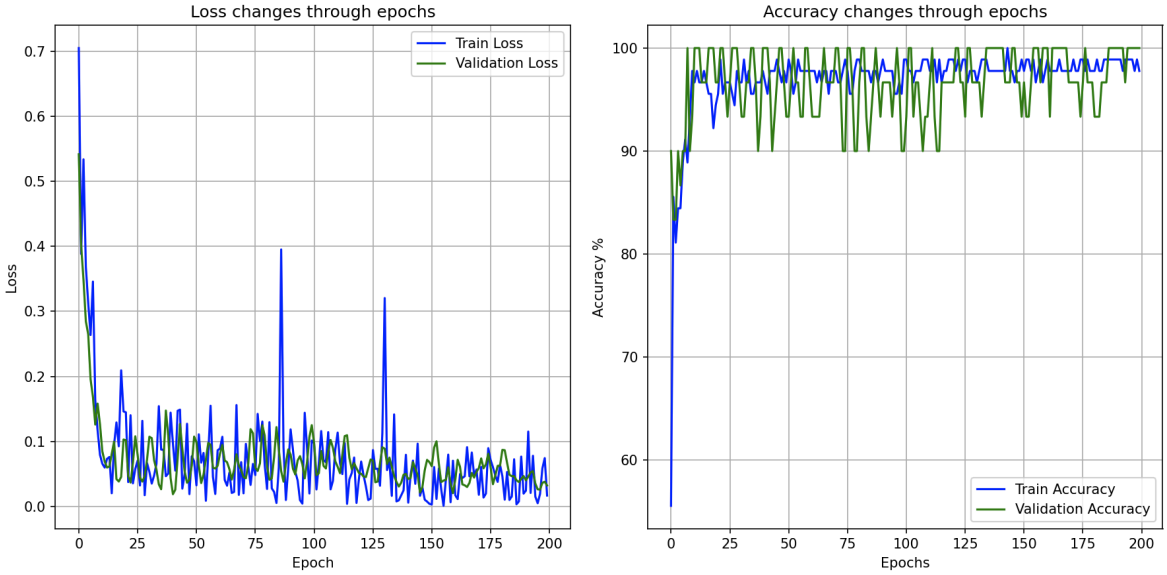
Observing impact of batch size on training process

Very small batch size: *Training_Set_Size / 18 (equals to 5 in current dataset)*



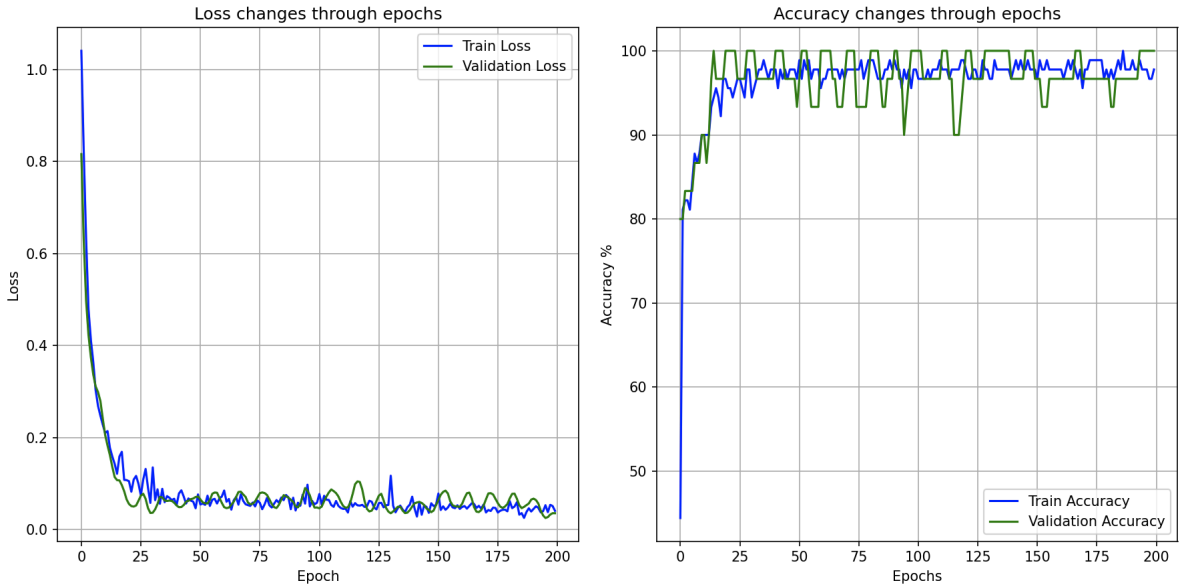
We can observe huge variations of loss function on the whole training process, as small batch sizes produce great instability because of how much data varies from batch to batch.

Medium batch size: *Training_Set_Size / 3 (equals to 30 in current dataset)*



We can see that the starting accuracy was lower compared to the small batch size, but after approximately 15 epochs it got to the 92+ percent values and was more stable in that range. Also loss function has lower values and less noise.

Big batch size: *Training_Set_Size (equals to 90 in current dataset)*



Loss and Accuracy curves are much more stable, because the model operates on the whole dataset at once, therefore computing gradients on entire dataset. However this requires much greater memory capacity and in greater applications, like Image processing, is not possible.